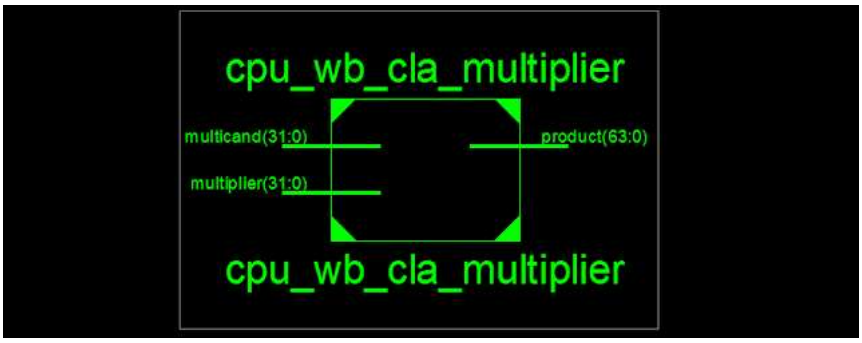
Verilog Code for Multiplier using Carry-Look-Ahead Adders

This project is to implement a parameterized multiplier using carry-look-ahead adders in Verilog. The Verilog code for the multiplier is provided.

Users can change the number of bits of the multiplier by modifying the predefined parameters. The parameters such as MULTICAND_WID and MULTIPLIER_WID are to define the number of bits of the multiplicand and multiplier, and when we want to change the number of bits, just change these parameters and re-synthesize or simulate.



Verilog code for Carry-Look-Ahead-Adder

```
`timescale 1ns/1ps
`define DELAY #10
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
// Verilog code for carry look-ahead adder
module cpu_wb_cla_adder (in1, in2, carry_in, sum, carry_out);
parameter DATA_WID = 32;

input [DATA_WID - 1:0] in1;
input [DATA_WID - 1:0] in2;
input carry_in;
output [DATA_WID - 1:0] sum;
output carry_out;

//assign {carry_out, sum} = in1 + in2 + carry_in;

wire [DATA_WID - 1:0] gen;
wire [DATA_WID - 1:0] pro;
wire [DATA_WID:0] carry_tmp;

genvar j, i;
generate
 //assume carry_tmp in is zero
 assign carry_tmp[0] = carry_in;

 //carry generator
 for(j = 0; j < DATA_WID; j = j + 1) begin: carry_generator
 assign gen[j] = in1[j] & in2[j];
 assign pro[j] = in1[j] | in2[j];
 assign carry_tmp[j+1] = gen[j] | pro[j] & carry_tmp[j];
 end

 //carry out
 assign carry_out = carry_tmp[DATA_WID];

 //calculate sum
 //assign sum[0] = in1[0] ^ in2 ^ carry_in;
 for(i = 0; i < DATA_WID; i = i+1) begin: sum_without_carry
 assign sum[i] = in1[i] ^ in2[i] ^ carry_tmp[i];
 end
```

```
endgenerate
endmodule
```

Verilog Testbench code for Carry-Look-Ahead Adder

```
 module cla_adder_tb();
parameter DATA_WID = 16;
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
// Verilog testbench code for carry look ahead adder
 reg carry_in; // To cla1 of cla_adder.v
 reg [DATA_WID-1:0] in1; // To cla1 of cla_adder.v
 reg [DATA_WID-1:0] in2; // To cla1 of cla_adder.v

// fpga4student.com FPGA projects, Verilog projects, VHDL projects // /*AUTOWIRE*/
 wire carry_out; // From cla1 of cla_adder.v
 wire [DATA_WID-1:0] sum; // From cla1 of cla_adder.v

// fpga4student.com FPGA projects, Verilog projects, VHDL projects //
cla_adder cla1(/*AUTOINST*/
        //  Outputs
         .sum (sum[DATA_WID-1:0]),
         .carry_out (carry_out),
        // // Inputs
         .in1 (in1[DATA_WID-1:0]),
         .in2 (in2[DATA_WID-1:0]),
         .carry_in (carry_in));

 initial begin
  in1 = 16'd0;
  in2 = 16'd0;
  carry_in = 1'b0;
  end
 // fpga4student.com FPGA projects, Verilog projects, VHDL projects
 initial begin
  #(`DELAY)
  #(`DELAY)  in1 = 16'd10;
  #(`DELAY)  in1 = 16'd20;
  #(`DELAY)  in2 = 16'd10;
  #(`DELAY)  in2 = 16'd20;
  #(`DELAY)  in2 = 16'd0;
  #(`DELAY*3)  in1 = 16'hFFFF; in2 = 16'hFFFF;
  #(`DELAY*3)  in1 = 16'h7FFF; in2 = 16'hFFFF;
  #(`DELAY*3)  in1 = 16'hBFFF; in2 = 16'hFFFF;
  end
 endmodule
```

Finally, the Verilog and testbench code for the parameterized carry look-ahead multiplier.

Verilog code for the Multiplier using carry-look-ahead adders:

```
`timescale 1ns/1ps
`define DELAY 10
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for multiplier using carry-look-ahead adders
module cpu_wb_cla_multiplier (multicand, multiplier, product);
parameter MULTICAND_WID = 32;
parameter MULTIPLIER_WID = 32;

input [MULTICAND_WID-1:0] multicand;
input [MULTIPLIER_WID-1:0] multiplier;
output [(MULTICAND_WID + MULTIPLIER_WID - 1):0] product;

wire [MULTICAND_WID - 1:0] multicand_tmp [MULTIPLIER_WID-1:0];
wire [MULTICAND_WID - 1:0] product_tmp [MULTIPLIER_WID-1:0];
wire [MULTIPLIER_WID -1:0] carry_tmp;
// fpga4student.com FPGA projects, Verilog projects, VHDL projects genvar i, j;
generate
 //initialize values
 for(j = 0; j < MULTIPLIER_WID; j = j + 1) begin: for_loop_j
 assign multicand_tmp[j] =  multicand & {MULTICAND_WID{multiplier[j]}};
 end

 assign product_tmp[0] = multicand_tmp[0];
 assign carry_tmp[0] = 1'b0;
```

```
 assign product[0] = product_tmp[0][0];
 // fpga4student.com FPGA projects, Verilog projects, VHDL projects
 for(i = 1; i < MULTIPLIER_WID; i = i + 1) begin: for_loop_i
 cpu_wb_cla_adder #(.DATA_WID(MULTIPLIER_WID)) add1 (
     // Outputs
     .sum(product_tmp[i]),
     .carry_out(carry_tmp[i]),
     // Inputs
   .carry_in(1'b0),
     .in1(multicand_tmp[i]),
     .in2({carry_tmp[i-1],product_tmp[i-1][31-:31]}));
 assign product[i] = product_tmp[i][0];
 end //end for loop
 assign product[(MULTIPLIER_WID+MULTIPLIER_WID-1):MULTIPLIER_WID] = {carry_tmp[MUL
endgenerate
endmodule
```

Verilog testbench code for the Multiplier:

```
// Verilog project: Verilog code for multiplier using carry look ahead adder
// fpga4student.com FPGA projects, Verilog projects, VHDL projects
module cla_multiplier_tb();
 parameter MULTICAND_WID = 32;
 parameter MULTIPLIER_WID = 32;
// fpga4student.com FPGA projects, Verilog projects, VHDL projects // /*AUTOREGINP
 // Beginning of automatic reg inputs (for undeclared instantiated-module inputs)
 reg [MULTICAND_WID-1:0] multicand; // To mul1 of cla_multiplier.v
 reg [MULTIPLIER_WID-1:0]multiplier; // To mul1 of cla_multiplier.v
 // End of automatics

 /*AUTOWIRE*/
 // Beginning of automatic wires (for undeclared instantiated-module outputs)
 wire [(MULTICAND_WID+MULTIPLIER_WID-1):0]product;// From mul1 of cla_multiplier.v
 // End of automatics

 cpu_wb_cla_multiplier mul1(/*AUTOINST*/
     // // Outputs
      .product (product[(MULTICAND_WID+MULTIPLIER_WID-1):0]),
     // // Inputs
      .multicand (multicand[MULTICAND_WID-1:0]),
     .multiplier (multiplier[MULTIPLIER_WID-1:0]));

 //initial begin
 // multicand = 16'd0;
 // multiplier = 8'd0;
 // end
// fpga4student.com FPGA projects, Verilog projects, VHDL projects  integer i;
 initial begin
 for (i = 0; i < 30; i = i + 1) begin: W
  #(`DELAY) multicand = multicand + 1; multiplier = multiplier + 1;
   end

  #(`DELAY) //correct
  multicand = 32'h00007FFF;
  multiplier = 32'h0000007F;

  #(`DELAY) //correct
  multicand = 32'h00008000;
  multiplier = 32'h000000F0;

  #(`DELAY) //faila
  multicand = 32'h00008FF0;
  multiplier = 32'h000000F0;
```

```
  #(`DELAY) //correct
  multicand = 32'h00007FF0;
  multiplier = 32'h000000F7;

  #(`DELAY) //correct
  multicand = 32'h0000FFFF;
  multiplier = 32'h000000FF;
 end
// fpga4student.com FPGA projects, Verilog projects, VHDL projects  endmodule
```

Simulation results for the Verilog multiplier:

The Verilog code for the parameterized multiplier is synthesizable and can be implemented on FPGA for verification.

Recommended Verilog projects:

1. **What is an FPGA? How Verilog works on FPGA**
2. Verilog code for FIFO memory
3. Verilog code for 16-bit single-cycle MIPS processor
4. Programmable Digital Delay Timer in Verilog HDL
5. Verilog code for basic logic components in digital circuits
6. Verilog code for 32-bit Unsigned Divider
7. Verilog code for Fixed-Point Matrix Multiplication
8. Plate License Recognition in Verilog HDL
9. Verilog code for Carry-Look-Ahead Multiplier
10. Verilog code for a Microcontroller
11. Verilog code for 4x4 Multiplier
12. Verilog code for Car Parking System
13. Image processing on FPGA using Verilog HDL
14. How to load a text file into FPGA using Verilog HDL
15. Verilog code for Traffic Light Controller
16. Verilog code for Alarm Clock on FPGA
17. Verilog code for comparator design
18. Verilog code for D Flip Flop
19. Verilog code for Full Adder
20. Verilog code for counter with testbench
21. Verilog code for 16-bit RISC Processor
22. Verilog code for button debouncing on FPGA
23. How to write Verilog Testbench for bidirectional/ inout ports
24. Tic Tac Toe Game in Verilog and LogiSim
25. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)
26. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
27. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
28. Verilog code for Decoder
29. Verilog code for Multiplexers
30. N-bit Adder Design in Verilog
31. Verilog vs VHDL: Explain by Examples
32. Verilog code for Clock divider on FPGA
33. **How to generate a clock enable signal in Verilog**
34. **Verilog code for PWM Generator**
35. **Verilog coding vs Software Programming**

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.